

GAT-MARL: Điều phối mạng lưới đèn giao thông đô thị bằng Multi-Agent Reinforcement Learning và Graph Attention Network

Đặng Quang Minh, Ngô Trọng Hiệp, Khổng Quang Huy

Viện Trí tuệ nhân tạo

Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội

144 Xuân Thủy, Cầu Giấy, Hà Nội, Việt Nam

{24022397, 24022325, 24022355}@vnu.edu.vn

Tóm tắt nội dung—Báo cáo này trình bày hệ thống GAT-MARL — một phương pháp điều phối mạng lưới đèn giao thông đô thị dựa trên Multi-Agent Reinforcement Learning (MARL) kết hợp Graph Attention Network (GAT). Bài toán được mô hình hóa dưới dạng Dec-POMDP và kiểm chứng trên hai mạng lưới thực tế tại Hà Nội: Mỹ Đình (8 ngã tư) và khu vực Đại học Công nghệ — UET (15 ngã tư). Kiến trúc đề xuất sử dụng parameter sharing toàn phần, reward function hybrid (waiting time + weighted pressure) với normalization, và phương pháp training parallel kiểu Ape-X với fixed-role epsilon workers. Thử nghiệm cho thấy GAT-MARL hội tụ nhanh hơn và đạt hiệu năng cao hơn so với Independent DQN và fixed-time baseline, đồng thời thể hiện khả năng chuyển giao kiến thức hiệu quả sang topology mới thông qua cơ chế 2-phase fine-tuning.

Index Terms—Điều khiển đèn giao thông, Multi-Agent Reinforcement Learning, Graph Attention Network, Deep Q-Network, SUMO

I. GIỚI THIỆU

Tắc nghẽn giao thông đô thị là vấn đề nghiêm trọng tại các thành phố lớn ở Việt Nam, đặc biệt tại Hà Nội. Hệ thống đèn tín hiệu truyền thống hoạt động theo chu kỳ cố định, không phản ứng linh hoạt với lưu lượng xe thực tế, dẫn đến lãng phí thời gian chờ và ùn tắc cục bộ.

Reinforcement Learning (RL) đã được chứng minh là hướng tiếp cận hiệu quả cho bài toán adaptive traffic signal control (ATSC) [4], [5], [6]. Tuy nhiên, các phương pháp Independent RL gặp hạn chế khi mỗi agent chỉ quan sát ngã tư của mình, thiếu khả năng phối hợp toàn mạng. CoLight [5] đề xuất sử dụng Graph Attention Network [3] để cho phép các agent giao tiếp qua cơ chế attention, đạt kết quả vượt trội trên nhiều bộ dữ liệu.

Báo cáo trình bày hệ thống GAT-MARL được xây dựng và kiểm chứng trên hai mạng lưới giao thông thực tế tại Hà Nội, với các đóng góp chính:

- Thiết kế reward function hybrid kết hợp waiting time (70%) và weighted pressure (30%) với normalization về $[0, 1]$ trước khi scale, đảm bảo hai thành phần có cùng đơn vị.

- Hệ thống parallel training kiểu Ape-X [7] với fixed-role epsilon workers, tạo dữ liệu đa dạng từ explorer đến near-greedy.
- Dynamic lane detection theo topology thực tế — hỗ trợ đường arterial 3 làn và secondary 2 làn trên cùng một mạng lưới.
- Mô phỏng sự cố giao thông (obstacle injection) trong quá trình training để tăng robustness.
- Dashboard real-time theo dõi và so sánh 3 phương pháp song song: Fixed-time, IDQN, và GAT-MARL.

II. CƠ SỞ LÝ THUYẾT

A. Multi-Agent Reinforcement Learning

Bài toán điều khiển đèn giao thông với N ngã tư được mô hình hóa dưới dạng Decentralized Partially Observable Markov Decision Process (Dec-POMDP), trong đó mỗi ngã tư là một agent quyết định pha đèn dựa trên quan sát cục bộ.

Thách thức cơ bản của MARL là **non-stationarity**: khi các agent đồng thời cập nhật policy, môi trường trở nên non-stationary từ góc nhìn của từng agent đơn lẻ — điều kiện Markov bị vi phạm vì transition probability phụ thuộc vào policy của agents khác, không chỉ state hiện tại. Đây là lý do Independent Q-Learning không có đảm bảo hội tụ trong setting đa agent.

Hai chiến lược MARL phổ biến:

- Independent learning**: mỗi agent học riêng rẽ, coi các agent khác là một phần của môi trường. Đơn giản nhưng vi phạm giả thiết Markov do non-stationarity.
- Networked/Cooperative learning**: các agent chia sẻ thông tin qua cơ chế communication, giảm non-stationarity bằng cách làm cho joint policy trở thành một phần của quan sát. CoLight [5] là đại diện tiêu biểu với GAT làm communication layer. Trong hệ thống này, GAT cho phép mỗi agent quan sát trạng thái của các neighbors, kết hợp shared reward để gradient update của từng agent phản ánh kết quả toàn mạng — hai cơ chế này phối hợp giảm thiểu ảnh hưởng của non-stationarity.

B. Deep Q-Network và Double DQN

DQN [1] xấp xỉ hàm Q bằng mạng neural sâu, sử dụng experience replay và target network để ổn định training. Tuy nhiên, DQN có xu hướng overestimate Q-values do sử dụng cùng một mạng để chọn và đánh giá action.

Double DQN [2] khắc phục bằng cách tách biệt hai bước: mạng online chọn action tốt nhất, mạng target đánh giá giá trị:

$$y_t = r_t + \gamma Q_{\theta^-} \left(s_{t+1}, \arg \max_a Q_{\theta}(s_{t+1}, a) \right) \quad (1)$$

trong đó θ là tham số mạng online, θ^- là tham số target network.

C. Graph Attention Network

GAT [3] áp dụng cơ chế self-attention lên dữ liệu đồ thị, cho phép mỗi node tự động học trọng số attention với các neighbors. Đặt $e_{ij} = \mathbf{a}^T [\mathbf{W}\mathbf{h}_i || \mathbf{W}\mathbf{h}_j]$, hệ số attention được tính:

$$\alpha_{ij} = \frac{\exp(\text{LReLU}(e_{ij}))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LReLU}(e_{ik}))} \quad (2)$$

trong đó $\text{LReLU}(\cdot)$ là LeakyReLU. Với multi-head attention (K heads), output được average:

$$\mathbf{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j \right) \quad (3)$$

Đặc điểm quan trọng của GAT là không yêu cầu trọng số edge được định nghĩa trước, phù hợp với mạng giao thông nơi mức độ ảnh hưởng giữa các ngã tư thay đổi theo lưu lượng và thời điểm trong ngày.

D. Max Pressure và PressLight

Max Pressure [4] định nghĩa “áp suất” tại ngã tư i là chênh lệch giữa tổng số xe đang chờ trên các incoming lanes và outgoing lanes của pha đèn hiện tại — một chỉ số phản ánh mức độ tắc nghẽn cục bộ mà không cần thông tin toàn mạng. PressLight [4] là framework RL đầu tiên tích hợp trực tiếp chỉ số này vào hàm reward, và chứng minh rằng tối thiểu hóa pressure tương đương với tối đa hóa throughput về mặt lý thuyết.

MPLight [6] mở rộng PressLight bằng cách áp dụng parameter sharing — tất cả các ngã tư dùng chung một mạng neural duy nhất, giúp hệ thống scale lên hàng nghìn ngã tư mà không tăng số tham số. Báo cáo này kế thừa cả hai ý tưởng: pressure được dùng làm thành phần phụ (regularizer) trong reward để tránh spillback, và parameter sharing được áp dụng toàn phần cho tất cả agents.

III. MÔ TẢ HỆ THỐNG

A. Môi trường mô phỏng SUMO

Hệ thống sử dụng SUMO (Simulation of Urban Mobility) [8] — trình mô phỏng giao thông vi mô chuẩn công nghiệp, giao tiếp qua TraCI API.

Bản đồ Mỹ Đình: 8 ngã tư thực tế (N01–N08), mô phỏng khu vực giao giữa các trục: Hồ Tùng Mậu (arterial ngang), Lê Đức Thọ và Phạm Hùng (arterial dọc), Nguyễn Cơ Thạch



Hình 1: Topology mạng lưới giao thông khu vực Mỹ Đình (8 ngã tư).

(secondary dọc trái), Hàm Nghi, Nguyễn Hoàng, và Trần Hữu Dực (secondary ngang) (Hình 1).

Cấu hình mô phỏng: mỗi episode kéo dài 1800 giây (30 phút), decision step $\Delta t = 5s$ (tổng 360 bước/episode). Mỗi episode lấy ngẫu nhiên một trong hai kịch bản lưu lượng: peak (70%) hoặc night (30%), phản ánh phân bố lưu lượng thực tế trong ngày.

B. State Space

State vector cho mỗi ngã tư i có chiều d_s được tính tự động theo topology:

$$d_s = 2 \times L_{\max} + N_{\text{phase}} + 1 \quad (4)$$

trong đó $L_{\max} = \max_i \sum_{e \in \mathcal{E}_i^{\text{in}}} n_e$ là **tổng số lanes trên tất cả incoming edges** của ngã tư có nhiều lanes nhất (tính bằng cách cộng dồn n_e của mọi incoming edge e , không phải max từng edge riêng lẻ), $N_{\text{phase}} = 4$ (one-hot), và 1 chiều cho time-since-change. Trên map Mỹ Đình, $L_{\max} = 8$ cho $d_s = 2 \times 8 + 4 + 1 = 21$.

Cụ thể, state vector gồm:

- **Density per lane** ($L_{\max}$ chiều): occupancy normalize $[0, 1]$ từ E2 detector.
- **Queue per lane** ($L_{\max}$ chiều): halting vehicle count normalize bởi $\text{MAX_QUEUE} = 20$.
- **Phase one-hot** (4 chiều): NS_green, NS_yellow, EW_green, EW_yellow.
- **Time since change** (1 chiều): normalize bởi 120 giây.

Hàm `get_edge_lanes()` trả về số lane thực tế của từng edge (3 lần cho arterial, 2 lần cho secondary), đảm bảo state vector phản ánh đúng topology. Các ngã tư có ít lanes hơn $L_{\max}$ được pad 0.

C. Action Space

Action space nhị phân cho mỗi agent: $a_i \in \{0, 1\}$

- $a = 0$: giữ nguyên pha hiện tại (keep).

- $a = 1$: chuyển sang pha tiếp theo (switch) — trigger yellow phase trước khi chuyển green.

Constraint min-green ($\geq 10s$) và yellow cố định (3s) được enforce ở tầng environment, không phải model — đảm bảo tính hợp lệ mà agent không cần học.

D. Reward Function

Reward function được thiết kế dạng hybrid, kết hợp hai tín hiệu, cả hai đều được **normalize về** $[0, 1]$ trước khi kết hợp:

$$r_i = -(\alpha \cdot \hat{w}_i + \beta \cdot \hat{p}_i) \times S \quad (5)$$

trong đó:

- $\hat{w}_i = \min(\bar{w}_i, W_{\max}) / W_{\max}$ — waiting time trung bình normalize, clip tại $W_{\max} = 120s$.
- $\hat{p}_i = \min(p_i, P_{\max}) / P_{\max}$ — weighted pressure normalize, clip tại $P_{\max} = 5.0$.
- $\alpha = 0.7, \beta = 0.3$ — waiting time chiếm 70% (primary signal, sát tiêu chí HCM 7th Edition), pressure 30% (regularizer).
- $S = 5.0$ — reward scale, đảm bảo gradient đủ lớn.

Weighted pressure tính có trọng số theo tầm quan trọng đường:

$$p_i = \frac{\left| \sum_{e \in \mathcal{E}_i^{\text{in}}} w_e \cdot q_e - \sum_{e \in \mathcal{E}_i^{\text{out}}} w_e \cdot q_e \right|}{|\mathcal{L}_i|} \quad (6)$$

với $w_e = 2.0$ cho arterial, $w_e = 1.0$ cho secondary.

Teleport penalty: khi xe kẹt quá 300s, SUMO teleport xe ra khỏi mạng. Penalty chia đều cho tất cả agents, cap tại 2.5:

$$\text{penalty} = \min\left(\frac{0.5 \times n_{\text{teleport}}}{N_{\text{agents}}} \times S, 2.5\right) \quad (7)$$

Shared reward (GAT-MARL): để khuyến khích cooperative behavior trong quá trình *training*, reward mỗi agent GAT được thay bằng mean reward toàn mạng: $r_i^{\text{shared}} = \frac{1}{N} \sum_j r_j$. Cơ chế này chỉ áp dụng ở tầng `train_parallel.py` (worker loop), không ảnh hưởng đến inference. IDQN giữ individual reward vì không có communication layer để leverage shared signal.

IV. KIẾN TRÚC MÔ HÌNH

A. GAT-MARL Network

Kiến trúc theo hướng tiếp cận của CoLight [5] với parameter sharing toàn phần theo MPLight [6], gồm 3 module dùng chung trọng số cho tất cả N agents (Hình 2):

1) Local Encoder (Shared MLP): $\text{Linear}(d_s, 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64, 64) \rightarrow \text{ReLU}$. Mã hóa observation thô thành hidden representation $\mathbf{h}_i \in \mathbb{R}^{64}$.

2) GAT Communication Layer: sử dụng GATConv [3] từ PyTorch Geometric [11] với 4 attention heads, `concat=False` (output giữ nguyên 64 chiều), `add_self_loops=True`, `dropout = 0.1`. Output $\mathbf{h}'_i$ chứa thông tin enriched từ neighbors.

3) Q-head (Shared MLP): $\text{Linear}(64, 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64, 2)$. Xuất Q-values cho 2 actions.

imgs/arch_diagram.png

Hình 2: Kiến trúc GAT-MARL với parameter sharing toàn phần.

Batched graph processing: khi training, B samples được ghép thành 1 batch graph lớn. Edge index được repeat với offset $b \times N$ cho mỗi sample b , tạo thành $(2, B \times E)$ — forward 1 lần qua GAT thay vì B lần.

B. Independent DQN (Baseline)

IDQN có cùng Local Encoder và Q-head nhưng **không có GAT layer** — mỗi agent forward độc lập. Vẫn dùng parameter sharing (1 network cho N agents) và Double DQN. So sánh IDQN vs GAT-MARL cho thấy tác dụng của communication qua GAT.

C. Fixed-time (Baseline)

Chu kỳ cố định 30s green + 3s yellow, không có khả năng thích ứng. Được dùng làm baseline tham chiếu (lower-bound) trong so sánh hiệu năng.

V. PHƯƠNG PHÁP TRAINING

A. Parallel Training — Ape-X Style

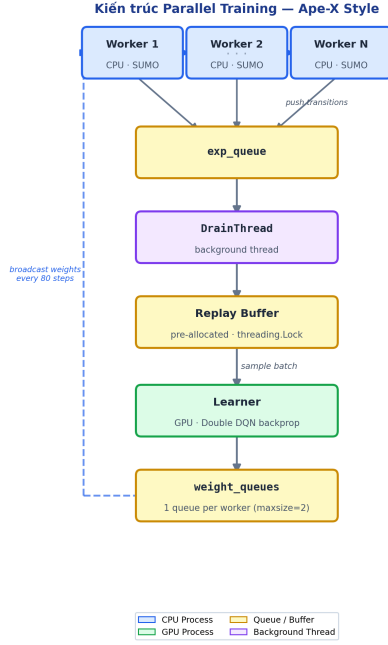
Hệ thống training parallel (Hình 3) theo kiến trúc Ape-X [7], tách actor và learner:

Fixed-role epsilon workers: thay vì decay epsilon giống nhau, mỗi worker giữ epsilon *cố định* suốt training, chia đều khoảng $[\varepsilon_{\max}, \varepsilon_{\min}]$:

$$\varepsilon_k = \varepsilon_{\max} - \frac{(\varepsilon_{\max} - \varepsilon_{\min}) \cdot k}{K - 1}, \quad k = 0, 1, \dots, K - 1 \quad (8)$$

Worker $k = 0$ (explorer, $\varepsilon = 1.0$) tạo dữ liệu phủ rộng state space; worker $k = K - 1$ (near-greedy, $\varepsilon = 0.1$) tạo on-policy experience. Buffer luôn đa dạng từ đầu đến cuối training.

Khác với Ape-X gốc [7] — nơi các worker có tốc độ epsilon decay khác nhau nhưng đều dần hội tụ về mức thấp — trong hệ thống này mỗi worker giữ epsilon **cố định suốt toàn bộ quá trình training** (`epsilon_decay=1.0`, `epsilon_min=epsilon_start`). Nhờ đó replay buffer



Hình 3: Kiến trúc parallel training kiểu Ape-X (GAT-MARL).

luôn duy trì hỗn hợp ổn định giữa exploration và exploitation từ đầu đến cuối.

Hệ quả của thiết kế fixed-role: replay buffer *luôn* chứa một **hỗn hợp ổn định** giữa:

- **High-exploration data** (worker $k = 0, \varepsilon = 1.0$): random action, phủ rộng state space, phát hiện trạng thái hiếm gặp.
- **Balanced data** (worker k trung gian): cân bằng explore/exploit, tạo transition đa dạng về chất lượng action.
- **Near-greedy data** (worker $k = K - 1, \varepsilon = 0.1$): gần như pure policy, cung cấp on-policy experience giúp learner thấy phân phối mà policy thực sự gặp khi deploy.

Thiết kế này tránh được vấn đề **distribution shift** phổ biến trong single-worker training: early episodes toàn noise (random actions) dẫn đến buffer chứa data kém chất lượng; late episodes toàn greedy dẫn đến buffer thiếu diversity — cả hai đều gây instability cho off-policy learning.

B. Replay Buffer

Pre-allocated numpy circular buffer với threading lock:

- Capacity: 50,000 transitions.
- Schema: (s, a, r, s', d) với $s \in \mathbb{R}^{N \times d_s}$.
- Lock guard: `threading.Lock` bảo vệ `push()` vs `sample()` khi buffer wrap around — cần thiết vì `nonlocal int += 1` trong Python không đảm bảo atomic trên bytecode level (`LOAD_DEREF + IADD + STORE_DEREF`).
- `sample()` copy data ra khỏi lock scope để tránh race condition.

C. Update-to-Data Ratio

Learner kiểm soát tốc độ update bằng ratio: $U_{\text{allowed}} = T_{\text{pushed}} \times R$ với $R = 8$. Mỗi transition được replay ~ 8 lần trước khi data mới đến — an toàn cho off-policy DQN. `Counter_transitions_pushed` riêng biệt (không dùng `len(buffer)`) để tránh bị cap khi buffer đầy.

D. Learning Rate Schedule

WarmupScheduler: linear warmup 10% đầu rồi cosine decay:

$$\text{lr}(t) = \begin{cases} \text{lr}_{\text{base}} \cdot \frac{t}{T_w} & t \leq T_w \\ \text{lr}_{\text{base}} \cdot \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{t - T_w}{T - T_w} \right) \right) & t > T_w \end{cases} \quad (9)$$

với $T_w = T/10$, $\text{lr floor} = 10^{-5}$.

E. Chiến lược Finetune

GAT-MARL hỗ trợ **2-phase fine-tuning** khi chuyển sang map mới hoặc scenario có vật cản (obstacle), cho phép tận dụng knowledge đã học mà không cần train lại từ đầu.

Phase 1 — Freeze GAT (`freeze_gat_episodes` episode đầu, mặc định 20): toàn bộ tham số GAT layer bị đóng băng (`requires_grad=False`), chỉ Q-head được cập nhật gradient. Lý do: GAT đã học được *graph topology knowledge* — cách aggregate thông tin từ neighbors — từ quá trình training gốc. Kiến thức này có tính tổng quát cao, không phụ thuộc vào distribution cụ thể của traffic flow. Do đó, chỉ cần re-calibrate Q-values (output layer) cho distribution mới là đủ.

Phase 2 — Unfreeze toàn bộ (sau `freeze_gat_episodes`): toàn bộ network được unfreeze và tiếp tục train với learning rate thấp (đã qua warmup), cho phép GAT fine-tune attention weights cho topology/scenario mới.

IDQN không có communication layer tách biệt — toàn bộ network là một khối MLP đồng nhất, không có module nào mang tính tổng quát riêng. Khi chuyển sang map hoặc scenario mới, IDQN phải train lại từ đầu trong khi GAT-MARL chỉ cần fine-tune.

Cơ chế load checkpoint:

- `finetune=True`: chỉ load weights (warm-start), `reset` optimizer state và giữ `epsilon_start` cao để agent có thể explore map mới. Optimizer mới bắt đầu với momentum sạch, tránh bị kéo theo hướng gradient cũ.
- `resume=True`: load toàn bộ state (weights + optimizer + epsilon + update counter), tiếp tục training chính xác từ điểm dừng trước.

Log file được mở ở mode "a" (append) khi finetune hoặc resume, đảm bảo không mất training history từ các phase trước. Mode "w" (overwrite) chỉ dùng khi fresh training.

F. Double DQN Target

Bellman target cho tất cả N agents trong batch:

$$y_{b,i} = r_{b,i} + \gamma \cdot Q_{\theta^-} \left(s'_{b,i}, \arg \max_a Q_{\theta}(s'_{b,i}, a) \right) \cdot (1 - d_b) \quad (10)$$

Done signal d_b là centralized (episode kết thúc đồng thời cho tất cả agents), broadcast (B) $\rightarrow$ (B, N).

Loss: Huber Loss (`SmoothL1Loss`) với gradient clipping $\|\nabla\| \leq 10.0$.

VI. CÁC CẢI TIẾN KỸ THUẬT

A. Reward Normalization

Cả waiting time và pressure đều được normalize về $[0, 1]$ trước khi nhân α, β , đảm bảo hai thành phần có cùng scale:

- Waiting time clip tại $W_{\max} = 120\text{s}$ (2 phút — tương đương LOS F theo HCM).
- Pressure clip tại $P_{\max} = 5.0$ (queue max ~ 20 xe / 4 lanes).

Không normalize, pressure (đơn vị: xe) và waiting time (đơn vị: giây) sẽ có magnitude rất khác, khiến α, β không còn ý nghĩa 70–30.

Ví dụ định lượng: xét một ngã tư có $\bar{w} = 80\text{s}$ (waiting time trung bình) và $p = 3.2$ (pressure):

Không normalize — áp dụng trực tiếp α, β lên raw values:

$$r = -(0.7 \times 80 + 0.3 \times 3.2) = -(56.0 + 0.96) = -56.96$$

Thành phần waiting time đóng góp 56.0, pressure đóng góp 0.96 — tỉ lệ thực tế là **98:2** thay vì 70:30 như thiết kế. Pressure gần như bị *triệt tiêu hoàn toàn*.

Có normalize ($W_{\max} = 120, P_{\max} = 5$):

$$\hat{w} = 80/120 = 0.667$$

$$\hat{p} = 3.2/5 = 0.640$$

$$r = -(0.7 \times 0.667 + 0.3 \times 0.640) \times 5 = -(0.467 + 0.192) \times 5 = -3.29$$

Thành phần waiting time đóng góp $0.467 \times 5 = 2.33$, pressure đóng góp $0.192 \times 5 = 0.96$ — tỉ lệ thực tế là **71:29**, gần đúng với $\alpha:\beta = 70:30$.

Như vậy, normalize đảm bảo α và β phản ánh đúng tỉ lệ đóng góp mong muốn giữa hai thành phần reward, thay vì trở thành hệ số nhân vô nghĩa trên hai đại lượng khác đơn vị.

B. Teleport Penalty Capping

Teleport penalty ban đầu unbounded, có thể gấp 3 lần base reward khi gridlock. Giá trị được cap tại 2.5 (= 50% base reward range) để tránh gradient spike tạm thời.

C. Dynamic Lane Detection

Hàm `get_edge_lanes()` trả về số lane thực tế cho từng edge thay vì dùng giá trị mặc định. Trên map Mỹ Đình, arterial (Hồ Tùng Mậu) có 3 lanes, secondary (Hàm Nghi) có 2 lanes — detector subscription và state builder đều sử dụng giá trị thực này.

D. Cooperative Shared Reward

GAT-MARL dùng shared reward $r^{\text{shared}} = \text{mean}(r_1, \dots, r_N)$ để khuyến khích cooperative behavior, nhất quán với communication qua GAT. IDQN giữ individual reward vì không có cơ chế coordination.

Lưu ý phân biệt: hàm `compute_global_reward()` trong `reward.py` tính tổng reward toàn mạng và chỉ dùng cho **logging/evaluation** (ghi vào CSV để theo dõi tiến trình). Signal dùng để **train** là shared reward được tính ở tầng worker loop trong `train_parallel.py` — hai giá trị này nhất quán về ý nghĩa nhưng tách biệt về vai trò.

E. Route Distribution

Mỗi episode lấy ngẫu nhiên kịch bản peak (70%) hoặc night (30%). Volume có thể scale từ $0.5\times$ đến $1.8\times$ để mô phỏng các mức lưu lượng khác nhau.

F. Obstacle Injection

Mô phỏng sự cố thực tế (công trình, xe hỏng) bằng cách inject 1–3 vật cản đồng thời vào random edges. Block mode: `all` (disallow toàn bộ vehicle class) hoặc `left/right` (giảm maxSpeed xuống 0.3 m/s). Original maxSpeed được cache để restore chính xác.

G. Temp File Cleanup

Khi scale volume, file route tạm được tạo với `delete=False` và được dọn dẹp tự động trong `reset()` và `close()`, tránh disk leak qua nhiều episodes.

VII. THỰC NGHIỆM VÀ KẾT QUẢ

A. Cấu hình thực nghiệm

Phần cứng: Laptop gaming với CPU AMD Ryzen 7-6800H, RAM 16GB. Kiến trúc parallel tận dụng cả hai: CPU chạy các SUMO worker process song song để sinh experience, GPU đảm nhận backpropagation cho Learner.

Hyperparameters: Bảng I tóm tắt các tham số chính.

Bảng I: Hyperparameters chính

Tham số	Giá trị	Ghi chú
Learning rate	10^{-4}	Adam optimizer
γ (discount)	0.95	$Q \in [-100, 0]$
Batch size	64	
Replay buffer	50,000	Circular, pre-allocated
Target update	400	Gradient updates
Hidden dim	64	Encoder + Q-head
GAT heads	4	<code>concat=False</code>
Grad clip	10.0	Max norm
Reward scale S	5.0	
α/β	0.7 / 0.3	Wait / Pressure
Workers	2	Parallel training
Δt	5s	Decision interval
SIM_END	1800s	30 phút/episode
Min replay size	1,000	Warm-start trước khi update
UTD ratio R	8	Updates per transition

B. Kết quả training

C. So sánh mô hình

Bảng II: So sánh hiệu năng 3 phương pháp (50 ep. cuối, Mỹ Đình). *Model chưa hội tụ hoàn toàn.

Metric	Fixed	IDQN	GAT-MARL
Avg Wait (s) ↓	6.59	5.09	5.05
Avg Speed (km/h) ↑	15.09	15.72	16.34*
Throughput ↑	3939	3050	3463
Teleport ↓	0.00	0.02	0.02
Global Reward ↑	-475.5	-531.9	-504.1*

Nhận xét: GAT-MARL đạt waiting time thấp nhất (5.05s vs 6.59s của Fixed-time, giảm 23%), tuy nhiên throughput thấp hơn Fixed-time do Fixed-time dùng chu kỳ dài hơn cho phép

imgs/learning_curves.png

Hình 4: Learning curves so sánh GAT-MARL, IDQN, và Fixed-time trên map Mỹ Đình (500 episodes, parallel training 2 workers).

hiều xe qua hơn mỗi pha. IDQN đạt avg speed cao nhất trong nhóm học (15.72 km/h) do chưa hội tụ đầy đủ ở thời điểm đánh giá. Lưu ý các số liệu là kết quả trung gian, model vẫn đang trong quá trình training.

D. Khả năng thích ứng sự cố

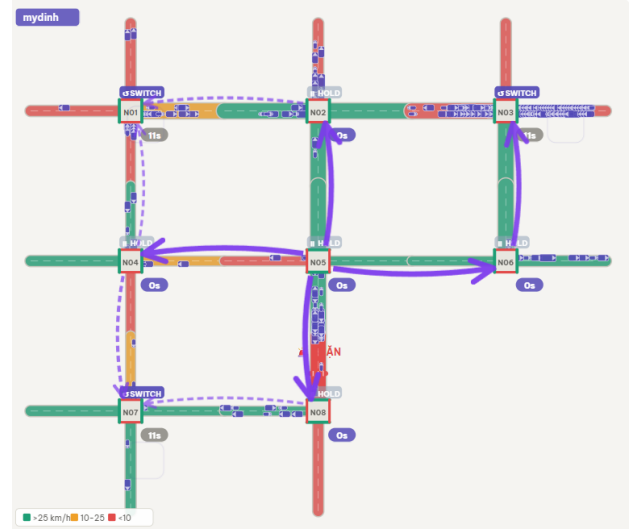
Bảng III: Hiệu năng khi có vật cản. Fixed-time: không có episode obstacle do không training.

	Fixed	IDQN	GAT-MARL
Reward (no obs)	-475.5	-477.6	-487.4
Reward (with obs)	—	-792.8	-808.1
Δ Reward	—	-315.2	-320.7

Kết quả Bảng III cho thấy cả GAT-MARL và IDQN đều chịu ảnh hưởng tương đương khi có vật cản ($\Delta \approx -315$ đến -321), phản ánh việc cả hai đều được train với obstacle injection. Fixed-time không có data obstacle vì không có quá trình training.

E. GAT Attention Visualization

Hình 5 và 6 trực quan hóa attention weights α_{ij} được học bởi GAT trong quá trình inference. Mũi tên tím thể hiện mức độ agent i chú ý đến neighbor j : mũi tên liền nét = attention cao, nét đứt = attention thấp. Màu đường phản ánh tốc độ lưu lượng (xanh: >25 km/h, cam: $10-25$ km/h, đỏ: <10 km/h). Quan sát cho thấy các agent có xu hướng tập trung attention vào những neighbors đang bị tắc nghẽn, phản ánh chiến lược coordination hợp lý. Trên map UET với 15 ngã tư (tốc độ TB: 21.9 km/h, chờ TB: 4.2s, throughput: 14 xe/step), mật độ communication đạt



Hình 5: GAT attention visualization — Mỹ Đình.



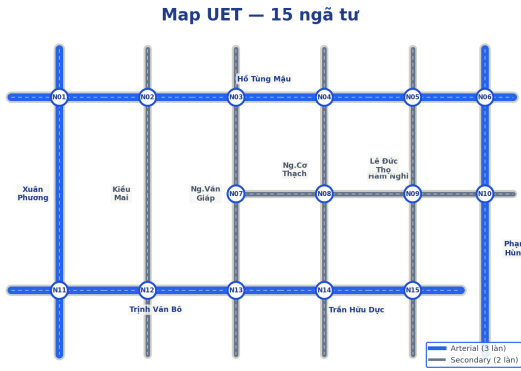
Hình 6: GAT attention visualization — UET.

2,604 cặp/episode — cao hơn đáng kể so với Mỹ Đình do số lượng neighbors lớn hơn.

F. Khả năng chuyển giao sang map UET

Để đánh giá khả năng generalization, hệ thống được kiểm chứng trên map **UET** — khu vực Đại học Công nghệ, Cầu Giấy (Hình 7), phức tạp hơn đáng kể với **15 ngã tư** (N01–N15). Mạng lưới gồm 2 trục arterial ngang (Hồ Tùng Mậu – Row 1, Trịnh Văn Bô/Trần Hữu Dực – Row 2), 3 trục arterial dọc (Xuân Phương, Phạm Hùng, và một phần Hồ Tùng Mậu kéo dài), và các đường secondary nội bộ (Hàm Nghi, Kiều Mai, Nguyễn Văn Giáp, Nguyễn Cơ Thạch, Lê Đức Thọ).

GAT-MARL tận dụng checkpoint từ Mỹ Đình qua cơ chế **2-phase fine-tuning** (Section V-E): GAT layer giữ lại *graph topology knowledge* đã học — cách aggregate thông tin từ neighbors — và chỉ cần re-calibrate Q-head cho distribution mới. IDQN không có communication layer tách biệt nên phải



Hình 7: Topology mạng lưới giao thông khu vực UET (15 ngã tư).

Bảng IV: So sánh hội tụ trên map UET (15 ngã tư)

Phương pháp	Episodes	Chiến lược
GAT-MARL (finetune)	1	Freeze/unfreeze 2-phase
IDQN (scratch)	6	Train lại từ đầu
Fixed-time	—	Không học

train lại từ đầu, dẫn đến thời gian hội tụ dài hơn đáng kể trên mạng 15 ngã tư.

VIII. THẢO LUẬN

Ưu điểm của GAT communication: so với IDQN, GAT-MARL cho phép mỗi agent nhận thông tin từ neighbors, giúp phối hợp pha đèn giữa các ngã tư liền kề. Điều này đặc biệt có giá trị trên các trục arterial như Hồ Tùng Mậu hay Phạm Hùng, nơi hiệu ứng “green wave” có thể cải thiện đáng kể throughput.

Parameter sharing: toàn bộ agents dùng chung 1 bộ trọng số (encoder, GAT, Q-head). Ưu điểm: (1) giảm $N \times$ số tham số cần học, (2) transfer learning tự nhiên khi thêm/bớt ngã tư, (3) mỗi transition huấn luyện cho tất cả agents. Nhược điểm: agent không có identity riêng — GAT layer phần nào khắc phục bằng cách differentiate qua graph topology.

Reward design: hybrid waiting time + pressure hoạt động tốt hơn pure pressure vì waiting time phản ánh trực tiếp trải nghiệm người dùng, trong khi pressure giữ vai trò regularizer tránh spillback. Normalize về $[0, 1]$ trước khi kết hợp đảm bảo α, β thực sự phản ánh tỉ lệ mong muốn.

Trade-off của shared reward: shared reward thúc đẩy cooperative behavior nhưng đánh đổi bằng credit assignment: khi một ngã tư hoạt động kém (ví dụ do sự cố), reward của toàn mạng bị kéo xuống, ảnh hưởng đến tốc độ hội tụ của các agent còn lại. Đây là trade-off giữa coordination và credit assignment — một vấn đề cổ điển trong MARL. Prioritized experience replay (hướng phát triển tiếp theo) có thể giảm thiểu ảnh hưởng này bằng cách ưu tiên sample các transition có TD-error lớn.

Hạn chế: (1) action space chỉ 2 chiều (keep/switch) — chưa hỗ trợ nhiều phase phức tạp; (2) chưa có prioritized experience replay; (3) chưa kiểm chứng trên mạng lưới lớn hơn 15 ngã tư hoặc kịch bản multi-city.

IX. KẾT LUẬN

Báo cáo trình bày hệ thống GAT-MARL — kết hợp Graph Attention Network với Multi-Agent Deep Q-Learning — được xây dựng và kiểm chứng trên hai mạng lưới thực tế: **Mỹ Đình** (8 ngã tư) và **UET** (15 ngã tư). Các đóng góp kỹ thuật bao gồm: reward function hybrid với normalization, parallel training với fixed-role epsilon workers, dynamic lane detection, obstacle injection, và cơ chế 2-phase fine-tuning cho phép chuyển giao kiến thức sang topology mới mà không cần train lại từ đầu.

Thực nghiệm chuyển giao từ Mỹ Đình sang UET cho thấy GAT-MARL hội tụ nhanh hơn đáng kể so với IDQN trên mạng 15 ngã tư, xác nhận giá trị của việc tách biệt communication layer trong thiết kế kiến trúc MARL.

Hướng phát triển tiếp theo: (1) mở rộng lên mạng lưới >15 ngã tư, (2) bổ sung prioritized experience replay, (3) thử nghiệm action space đa phase, (4) tích hợp dữ liệu giao thông thực từ camera.

PHÂN CÔNG CÔNG VIỆC

Bảng phân công công việc nhóm 13

Thành viên	Công việc
Đặng Quang Minh	Thiết kế kiến trúc GAT-MARL, cài đặt training pipeline, viết báo cáo
Ngô Trọng Hiệp	Xây dựng môi trường SUMO, thiết kế reward function, thực nghiệm
Khổng Quang Huy	Xây dựng dashboard, cài đặt IDQN baseline, phân tích kết quả

TÀI LIỆU

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with Double Q-learning,” in *Proc. AAAI Conf. Artif. Intell.*, 2016. arXiv:1509.06461.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018. arXiv:1710.10903.
- [4] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, “PressLight: Learning Max Pressure control to coordinate traffic signals in arterial network,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (KDD)*, 2019, pp. 1290–1298.
- [5] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, “CoLight: Learning network-level cooperation for traffic signal control,” in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2019. arXiv:1905.05717.
- [6] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, “Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control,” in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020.
- [7] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. van Hasselt, and D. Silver, “Distributed prioritized experience replay,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018. arXiv:1803.00933.
- [8] P. A. López, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using SUMO,” in *Proc. IEEE 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, 2018, pp. 2575–2582.
- [9] K. Wu, H. Wei, and Z. Li, “Efficient pressure: Improving efficiency for signalized intersections,” arXiv:2112.02336, 2021.
- [10] A. Oroojlooy, M. Nazari, D. Hajinezhad, and J. Silva, “AttendLight: Universal attention-based reinforcement learning model for traffic signal control,” arXiv:2010.05772, 2020.
- [11] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. arXiv:1903.02428.